Aarush Yadav 0827AL221001

Decision Tree - Heart

23/07/2026

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv(r"C:\Users\maila\OneDrive\Desktop\Mircrosoft_Training\Machne Learning\Decision_Tree and SVM\23july\heart.csv")
```

```
df.head()
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3	0
1	53	1	0	140	203	1	0	155	1	3.1	0	0	3	0
2	70	1	0	145	174	0	1	125	1	2.6	0	0	3	0
3	61	1	0	148	203	0	1	161	0	0.0	2	1	3	0
4	62	0	0	138	294	1	1	106	0	1.9	1	3	2	0

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
df.shape
```

```
(1025, 14)
```

```
print(df.columns)
```

```
Index(['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach',
       'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target'],
      dtype='object')
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1025 entries, 0 to 1024
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype  
---  ------    --------------  -----  
 0   age       1025 non-null   int64  
 1   sex       1025 non-null   int64  
 2   cp        1025 non-null   int64  
 3   trestbps  1025 non-null   int64  
 4   chol      1025 non-null   int64  
 5   fbs       1025 non-null   int64  
 6   restecg   1025 non-null   int64  
 7   thalach   1025 non-null   int64  
 8   exang     1025 non-null   int64  
 9   oldpeak   1025 non-null   float64
 10  slope     1025 non-null   int64  
 11  ca        1025 non-null   int64  
 12  thal      1025 non-null   int64  
 13  target    1025 non-null   int64  
dtypes: float64(1), int64(13)
memory usage: 112.2 KB
```

```
df.describe()
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpea
count	1025.000000	1025.000000	1025.000000	1025.000000	1025.00000	1025.000000	1025.000000	1025.000000	1025.000000	1025.00000
mean	54.434146	0.695610	0.942439	131.611707	246.00000	0.149268	0.529756	149.114146	0.336585	1.07151
std	9.072290	0.460373	1.029641	17.516718	51.59251	0.356527	0.527878	23.005724	0.472772	1.17505
min	29.000000	0.000000	0.000000	94.000000	126.00000	0.000000	0.000000	71.000000	0.000000	0.00000
25%	48.000000	0.000000	0.000000	120.000000	211.00000	0.000000	0.000000	132.000000	0.000000	0.00000
50%	56.000000	1.000000	1.000000	130.000000	240.00000	0.000000	1.000000	152.000000	0.000000	0.80000
75%	61.000000	1.000000	2.000000	140.000000	275.00000	0.000000	1.000000	166.000000	1.000000	1.80000
max	77.000000	1.000000	3.000000	200.000000	564.00000	1.000000	2.000000	202.000000	1.000000	6.20000

```
df.target
```

```
0       0
1       0
2       0
3       0
4       0
       ..
1020    1
1021    0
1022    0
1023    1
1024    0
Name: target, Length: 1025, dtype: int64
```

```
df.isnull().sum()
```

```
age         0
sex         0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
thalach     0
exang       0
oldpeak     0
slope       0
ca          0
thal        0
target      0
dtype: int64
```

```
df["ca"].fillna(df["ca"].mode()[0], inplace=True)
df["thal"].fillna(df["thal"].mode()[0], inplace=True)
```

```
df.isnull().sum()
```

```
age         0
sex         0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
thalach     0
exang       0
oldpeak     0
slope       0
ca          0
thal        0
target      0
dtype: int64
```

```
print("Target: ")
print("0 = No Heart Disease")
print("1 = Heart Disease")
```

```
Target:
0 = No Heart Disease
1 = Heart Disease
```

```
df.duplicated().sum()
```

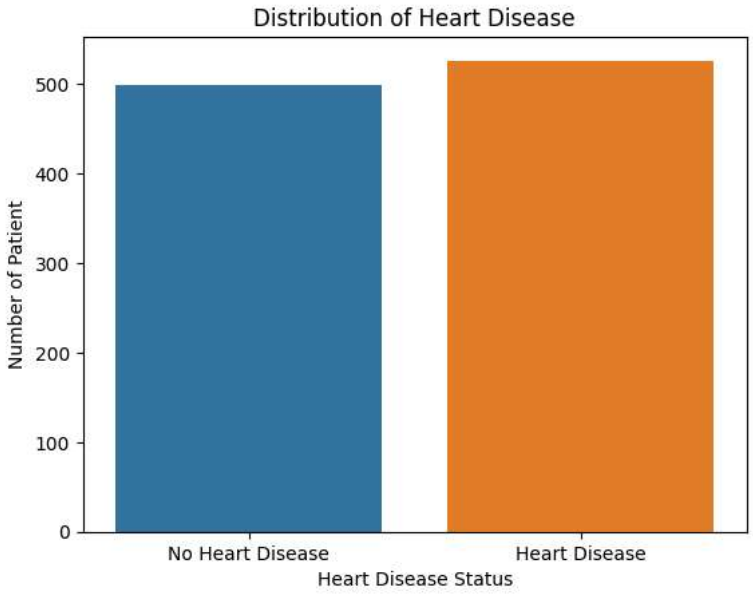
```
723
```

```
df['target'].value_counts() #with and without heart disease
```

```
1    526
0    499
Name: target, dtype: int64
```

```
sns.countplot(x = "target", data = df)

plt.xticks([0,1], ["No Heart Disease", "Heart Disease"])
plt.xlabel("Heart Disease Status")
plt.ylabel("Number of Patient")
plt.title("Distribution of Heart Disease")
plt.show()
```



splitting the dataset

```
X = df.drop("target", axis=1)
y = df["target"]
```

```
X.head()
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3
1	53	1	0	140	203	1	0	155	1	3.1	0	0	3
2	70	1	0	145	174	0	1	125	1	2.6	0	0	3
3	61	1	0	148	203	0	1	161	0	0.0	2	1	3
4	62	0	0	138	294	1	1	106	0	1.9	1	3	2

```
y.head()
```

```
0    0
1    0
2    0
3    0
4    0
Name: target, dtype: int64
```

train_test_split

```
#80 20 split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
print("Shape of X_train :", X_train.shape)
print("Shape of X_test  :", X_test.shape)

print("Shape of y_train :", y_train.shape)
print("Shape of y_test  :", y_test.shape)
```

```
Shape of X_train : (820, 13)
Shape of X_test  : (205, 13)
Shape of y_train : (820,)
Shape of y_test  : (205,)
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
model = DecisionTreeClassifier(criterion='entropy',random_state=42)
```

Train the model

```
model.fit(X_train, y_train)
```

```
DecisionTreeClassifier
DecisionTreeClassifier(criterion='entropy', random_state=42)
```

```
y_pred = model.predict(X_test)
```

Actual vs Predicted compare

```
comparison = pd.DataFrame({"Actual": y_test,"Predicted": y_pred})

comparison.head(10)
```

	Actual	Predicted
527	1	1
359	1	1
447	0	0
31	1	1
621	0	0
590	1	1
905	0	0
737	0	0
76	1	1
948	0	0

Calculate the accuracy

```
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.9853658536585366
```

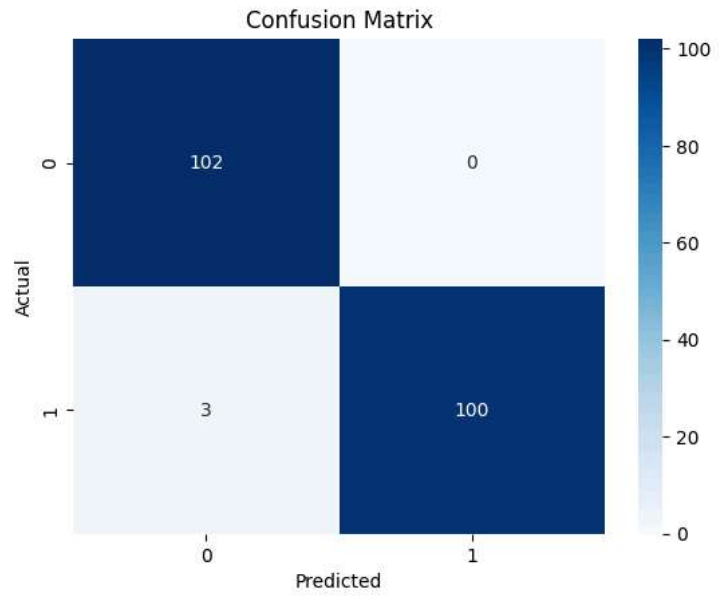
```
cm = confusion_matrix(y_test, y_pred)
print(cm)
```

```
[[102   0]
 [  3 100]]
```

```
sns.heatmap(
    cm,
    annot=True,
    cmap="Blues",
    fmt="d"
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")

plt.show()
```



Report

```
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.97      1.00      0.99       102
           1       1.00      0.97      0.99       103

    accuracy                           0.99       205
   macro avg       0.99      0.99      0.99       205
weighted avg       0.99      0.99      0.99       205
```

Predict a new patient

```
new_patient = [[55,1,2,130,250,0,1,150,0,1.2,2,0,2]]
prediction = model.predict(new_patient)
print(prediction)
```

```
[0]
C:\Users\maila\AppData\Local\Programs\Python\Python311\Lib\site-packages\sklearn\base.py:439: UserWarning: X does not have valid
  warnings.warn(
```

```
print("Predicted Class:", prediction[0])

if prediction[0] == 1:
    print("✅ The patient is likely to have Heart Disease.")
else:
    print("✅ The patient is unlikely to have Heart Disease.")
```

```
Predicted Class: 0
✅ The patient is unlikely to have Heart Disease.
```

```
if prediction[0] == 1:
    print("The patient is likely to have Heart Disease.")
else:
    print("The patient is unlikely to have Heart Disease.")
```

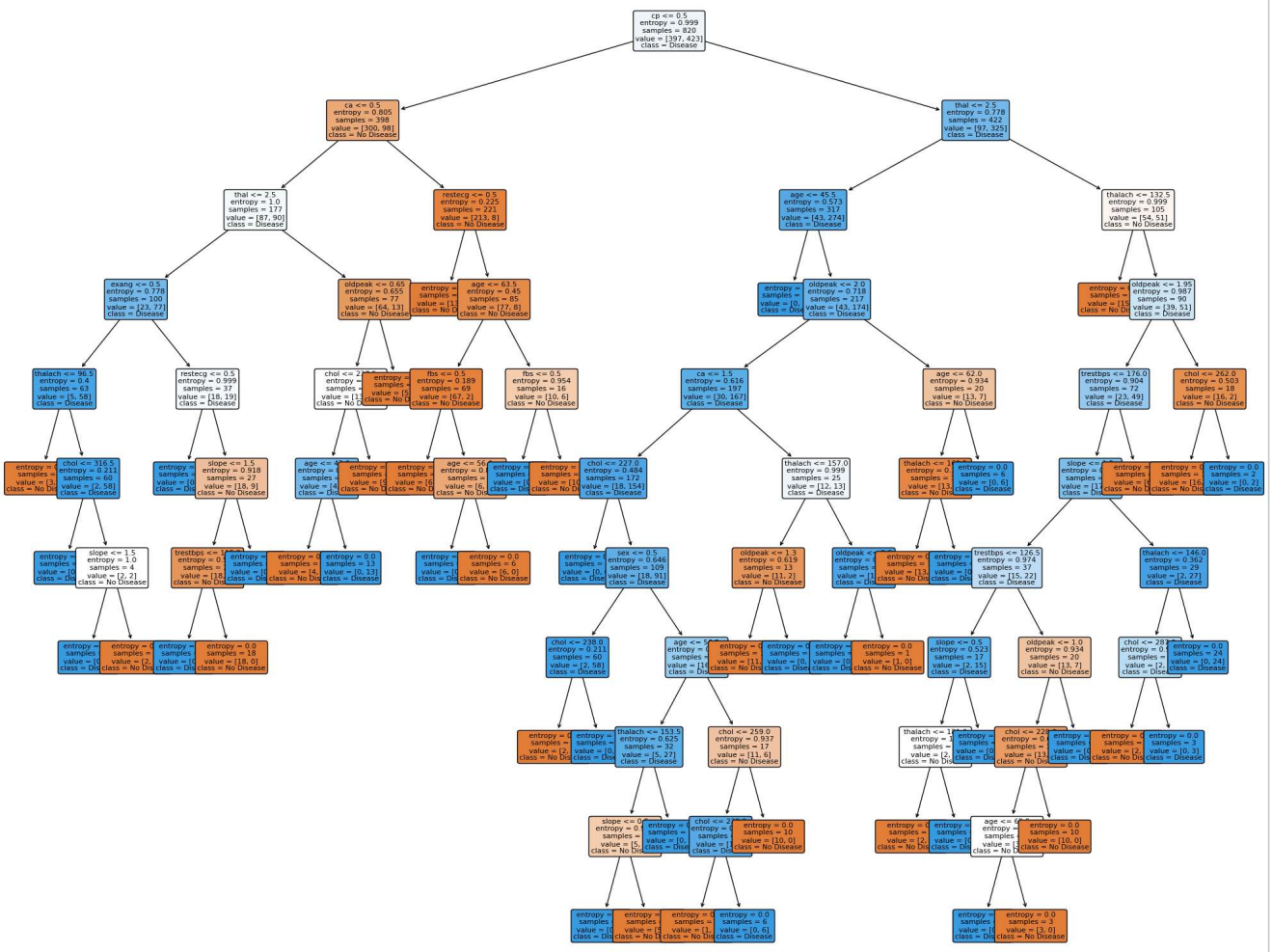
```
The patient is unlikely to have Heart Disease.
```

```
from sklearn.tree import plot_tree

plt.figure(figsize=(25,20))

plot_tree(
    model,
    feature_names=X.columns,
    class_names=["No Disease","Disease"],
    filled=True,
    rounded=True,
    fontsize=8
)

plt.show()
```




```
importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": model.feature_importances_
})

importance = importance.sort_values(
    by="Importance",
    ascending=False
)

importance
```

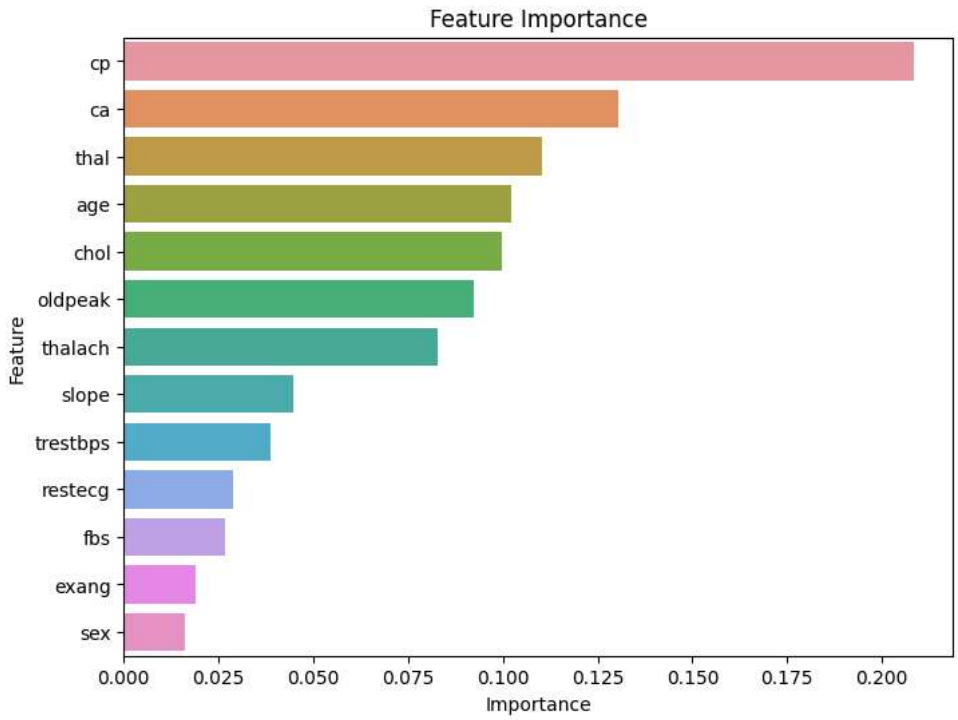
	Feature	Importance
2	cp	0.208321
11	ca	0.130591
12	thal	0.110385
0	age	0.102150
4	chol	0.099591
9	oldpeak	0.092290
7	thalach	0.082755
10	slope	0.044754
3	trestbps	0.038624
6	restecg	0.028758
5	fbs	0.026655
8	exang	0.019068
1	sex	0.016058

```
plt.figure(figsize=(8,6))

sns.barplot(
    x="Importance",
    y="Feature",
    data=importance
)

plt.title("Feature Importance")

plt.show()
```




```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)
```

```
accuracy = accuracy_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)

print("Accuracy :", round(accuracy, 4))
print("Precision:", round(precision, 4))
print("Recall   :", round(recall, 4))
print("F1 Score :", round(f1, 4))
```

```
Accuracy : 0.9854
Precision: 1.0
Recall   : 0.9709
F1 Score : 0.9852
```

```
print(f"Accuracy : {accuracy*100:.2f}%")
print(f"Precision: {precision*100:.2f}%")
print(f"Recall   : {recall*100:.2f}%")
print(f"F1 Score : {f1*100:.2f}%")
```

```
Accuracy : 98.54%
Precision: 100.00%
Recall   : 97.09%
F1 Score : 98.52%
```

```
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.97      1.00      0.99       102
           1       1.00      0.97      0.99       103

    accuracy                           0.99       205
   macro avg       0.99      0.99      0.99       205
weighted avg       0.99      0.99      0.99       205
```

```
print("\nInterpretation of Results")
print("-------------------------")
print(f"Accuracy : {accuracy*100:.2f}%")
print("→ Overall percentage of correct predictions.\n")

print(f"Precision: {precision*100:.2f}%")
print("→ When the model predicts Heart Disease, how often is it correct?\n")

print(f"Recall: {recall*100:.2f}%")
print("→ Out of all patients who actually have Heart Disease, how many did the model correctly identify?\n")

print(f"F1 Score: {f1*100:.2f}%")
print("→ A balance between Precision and Recall.")
```

```

Interpretation of Results
-------------------------
Accuracy : 98.54%
→ Overall percentage of correct predictions.
```